

IDLCV Project 1: Classification

Group 25

Atai Mozhdah
s224538

Crrara Alessandra
s252976

Lykouresis Nikolaos Dimitrios
s243174

Rueda Alberto
s253057

Tsilingeridis Emmanouil
s253154

1. Introduction

The goal of the project is to develop a model capable of identifying whether an image contains a hotdog (*hotdog class*) or not (*nothotdog class*). To achieve this, we started with the notebook and data set provided, implementing a base model employing convolutional neural networks, increased its complexity and added features, like the data augmentation or the batch normalization, to answer all the questions requested.

2. Architecture

The final architecture of the model used to classify the images consists of three main parts:

1. Convolutional layers: Three initial convolutional layers were used. In each of them, it begins with a convolutional layer where the number of features was increased, followed by a normalization, ReLU to introduce non-linearity, and MaxPool with which we reduced the dimensionality of the image. This approach is inspired by the VGG model.
2. Residual layers: two ResNets were introduced to leverage the identity mapping feature of these.
3. Fully connected: two fully connected layers. The first one to compress to a 128 dimension vector, ReLU to introduce non-linearity, a dropout with probability 0.35 to avoid overfitting and a final linear layer with a two-size output representing the two classes.

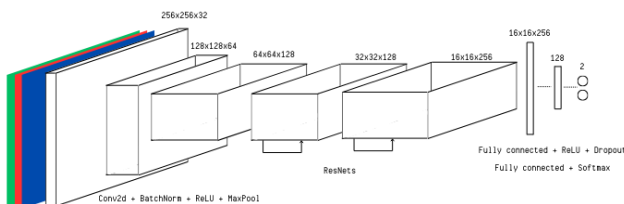


Figure 1. Architecture of the model

The initial architecture proposed consisted of the convolutional layers and the fully-connected. After adding the residual layers, the accuracy of the model increased 3% and the training time decreased notably.

3. Training of the model

The data set provided consisted of two folders for training and testing. The training data was divided into a training data set and a validation data set. This split was made by randomly taking 20% from the original training set.

The validation data set was used to avoid overfitting the model by implementing an early stopping. In this way, the parameters at which loss validation stops improving are saved. Figure 2 represents the accuracy, where it should be noted that there is a high oscillation mainly in the validation accuracy. This could be due to the small size of the validation set. Also, it is worth mention that the train accuracy is consistently less than the validation accuracy. One reason for this could be the use of dropout because during the calculation of the validation loss it is disabled. Figure 3 represents the training and validation loss, where we can see that the validation loss started to be very low.

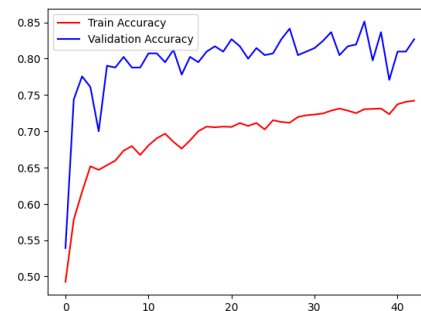


Figure 2. Accuracy on the training and validation set

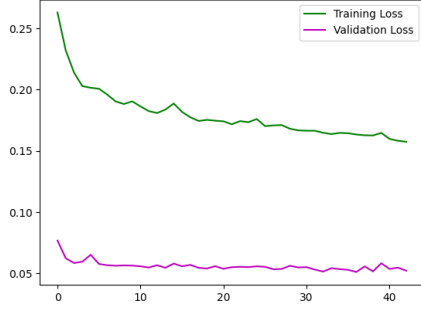


Figure 3. Training and validation loss

3.1. Optimizer and Loss Function

After experimenting with different optimization strategies, including plain *SGD*, *SGD* with momentum, and *Adam*, we observed distinct behaviors in convergence and generalization. While *Adam* achieved faster initial convergence and smoother loss reduction, it often resulted in less stable training and slightly weaker generalization. Plain *SGD* provided stable updates but converged more slowly and was more sensitive to learning rate tuning. In contrast, *SGD* with momentum offered a good compromise, combining the stability and robustness of *SGD* with faster convergence through accumulated gradient information. For these reasons, we selected *SGD* with momentum as it provided the best balance between convergence speed, stability, and generalization performance.

We also experimented with different loss functions. Initially, we used the *Negative Log-Likelihood (NLL)* loss with our first model, but later switched to the *Binary Cross-Entropy (BCE)* loss, which is more suitable for binary classification tasks and provides a better probabilistic interpretation for binary decisions.

4. Data Augmentation and Batch Normalization

In this section, we evaluate the efficiency of our models by following a stepwise approach. First, we test a simple model without applying either data augmentation or batch normalization. Next, we incorporate each technique individually, training one model with augmentation only and another with batch normalization only. Finally, we combine both techniques in a single model to assess whether their joint use leads to improved results. The results for each model configuration are presented in the table 1.

The experimental results indicate that batch normalization was the most influential factor in improving model performance. Compared to the simple baseline, batch normalization consistently yielded a 3–5% increase in validation accuracy and a 1–2% improvement in test accuracy, highlighting its effectiveness in stabilizing training and enhanc-

Model	Train	Validation	Test
Simple Model	79.1%	81.7%	77.4%
Augmentation Model	69.8%	81.5%	79.2%
Batch Normalized Model	79.3%	84.4%	78.7%
Combined Model	74.1%	85.1%	80.8%

Table 1. Performance comparison of different models.

ing generalization. Data augmentation alone provided only marginal benefits, with validation accuracy essentially unchanged and test accuracy increasing by about 2%. When combined with batch normalization, augmentation led to the best overall test performance (80.8%), but the incremental gains over batch normalization alone were relatively small. These findings suggest that while data augmentation contributes to robustness, batch normalization is the primary driver of improved performance.

5. Model Accuracy

Throughout the training process, the model showed a steady improvement in both training and validation accuracy. Initially, the network achieved around **51%** accuracy on both the training and validation sets, which indicates random-like performance at the start. As training progressed, accuracy increased consistently, reaching approximately **74.1%** on the training set and **85.1%** on the validation set before early stopping was triggered. This suggests that the model was able to generalize well to unseen data without significant overfitting. Finally, on the held-out test set, the network achieved a **Test Accuracy of 80.8%** with a corresponding test loss of **0.493**, confirming good generalization capability and balanced learning between training and validation phases.

In Figure 4, some examples of misclassified images from the test phase are shown. Several classification errors can be attributed to visual similarities between certain dishes and hotdogs. Images with sausage-like shapes, such as breaded foods, sandwiches, or elongated items, often lead to false positives, while hotdogs with unusual toppings or non-standard buns may be mistaken for non-hotdog images. Scenes featuring multiple foods or people can also distract the model, especially when the hot dog is partially visible or off-center. Moreover, close-up shots may highlight only textures or small portions of food, making it harder for the model to capture the full context for accurate classification.

6. Transfer Learning

We experimented with transfer learning by using EfficientNet-B0 pre-trained on ImageNet. The final classification layer was replaced with a two-class output corresponding to hotdog and not hotdog. This allowed us to leverage the pre-trained convolutional layers as general



Figure 4. Examples of failed classification during test phase

feature extractors while adapting the classifier to our specific task.

Compared to our baseline CNN trained from scratch, the transfer learning model converged faster and achieved higher accuracy obtaining almost 95% of accuracy. This demonstrates that transfer learning improves performance, especially when working with relatively small datasets, as it reduces overfitting and provides better generalization.

7. Saliency Maps

Our final CNN achieved around 80.8 % test accuracy, confirming good generalization. To interpret the model's decisions, we generated saliency maps.

Figure 5 shows a **Grad-CAM** visualization for a hotdog image. Grad-CAM highlights the discriminative regions in the convolutional feature maps that contribute most strongly to the classification. Compared to raw gradient-based saliency methods, Grad-CAM produces clearer and more interpretable heatmaps, as it localizes the important object regions rather than scattered pixel-level signals.

To complement this, we also applied **SmoothGrad**, which averages gradients over multiple noisy perturbations of the input. This produces clearer and more interpretable heatmaps.

For hotdog images, both Grad-CAM and SmoothGrad highlighted the hotdog region itself, showing that the network focused on meaningful features. For non-hotdog images, the maps emphasized the main object, such as a dog, indicating that the model captures discriminative regions

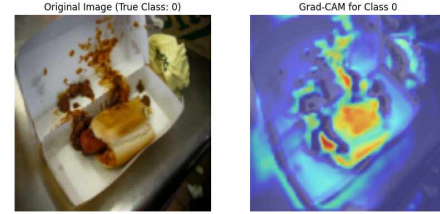


Figure 5. Hotdog image and its Grad-CAM saliency map

even when no hotdog is present.

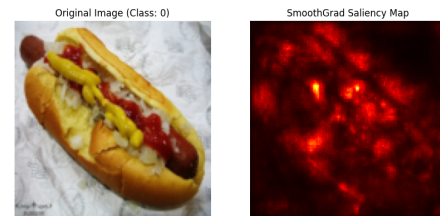


Figure 6. Hotdog image and its SmoothGrad saliency map

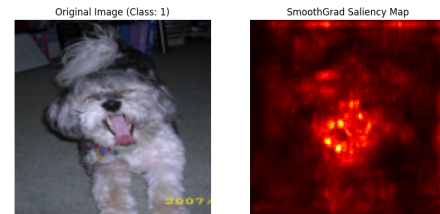


Figure 7. Non-hotdog image and its SmoothGrad saliency map

Figure 6 illustrates a correctly classified hotdog, while Figure 7 shows a non-hotdog example. These confirm that the model generally attends to relevant input regions, though saliency maps should be interpreted with caution due to their known limitations.

8. Declaration of the Use of AI Tools

We used AI tools to support our work in this project. Firstly, it was applied in the debugging process, where the tool assisted in identifying potential causes of errors and clarifying why certain parts of the code did not function as intended. This contributed to a more efficient resolution of technical issues and a deeper understanding of the underlying problems. In addition, they were used in the refinement of our written report, particularly in rephrasing and polishing the text to improve clarity, precision, and overall readability. In this way, the tool served as a supplementary aid that improved both the technical and communicative aspects of our work.